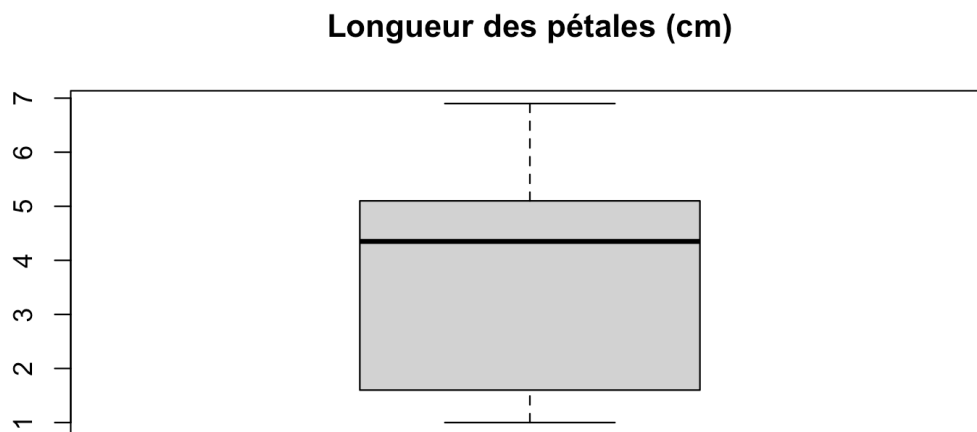


Solutionnaire du TP1

À votre tour

1. Boxplot (R de base)

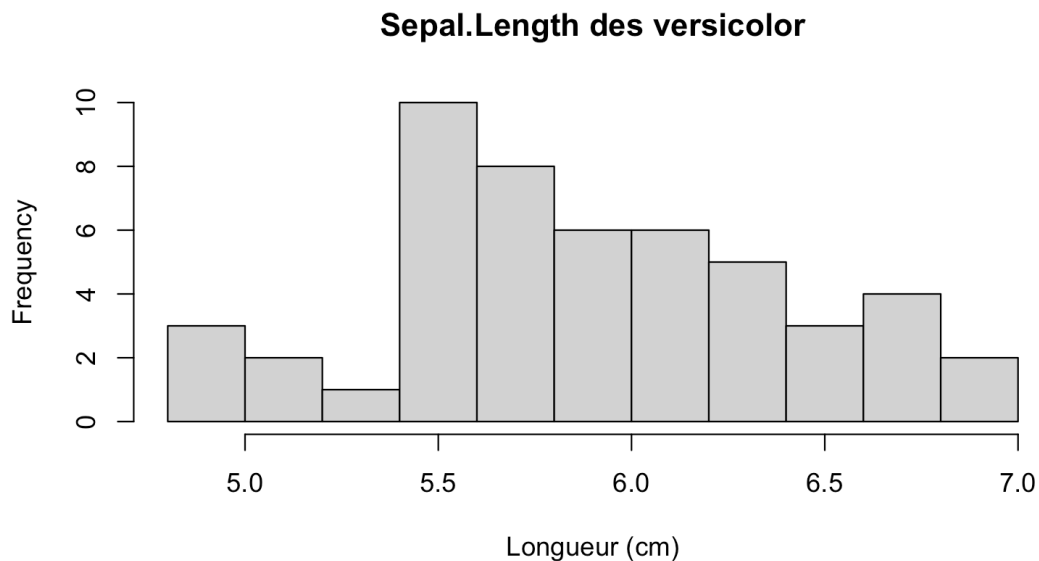
```
boxplot(iris$Petal.Length, main = "Longueur des pétales (cm)")
```



Une seule boîte, pour les 150 fleurs. La médiane est haute et la moustache basse très longue : les pétales courts des *setosa* tirent la distribution vers le bas. `boxplot(Petal.Length ~ Species, data = iris)` le montrerait.

2. Histogramme d'une espèce

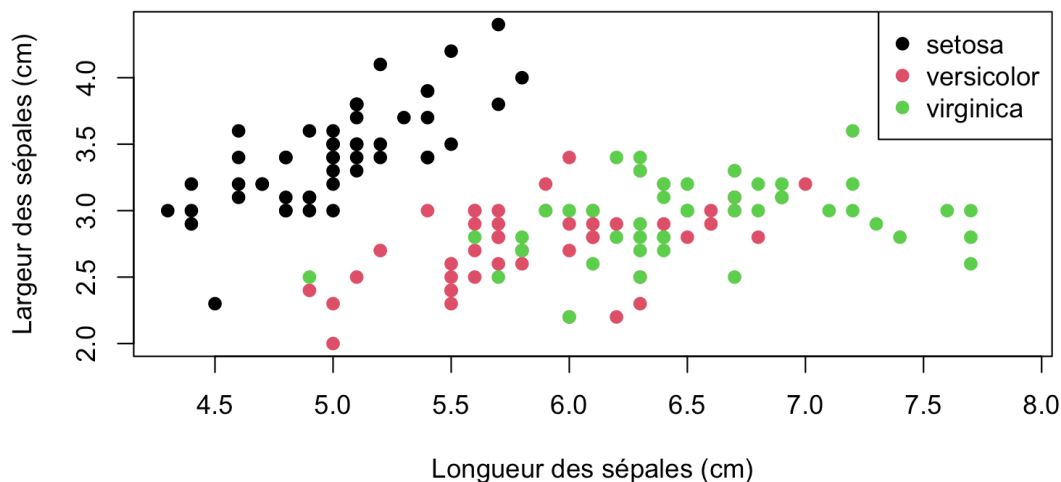
```
versicolor <- iris$Sepal.Length[iris$Species == "versicolor"]  
hist(versicolor, main = "Sepal.Length des versicolor", xlab = "Longueur (cm)", breaks = 1
```



Avec le *tidyverse*: `iris |> filter(Species == "versicolor") |> pull(Sepal.Length) |> hist()`.

3. Nuage de points avec une légende juste

```
plot(iris$Sepal.Length, iris$Sepal.Width, col = iris$Species, pch = 19,
     xlab = "Longueur des sépales (cm)", ylab = "Largeur des sépales (cm)")
legend("topright", legend = levels(iris$Species), col = 1:3, pch = 19)
```

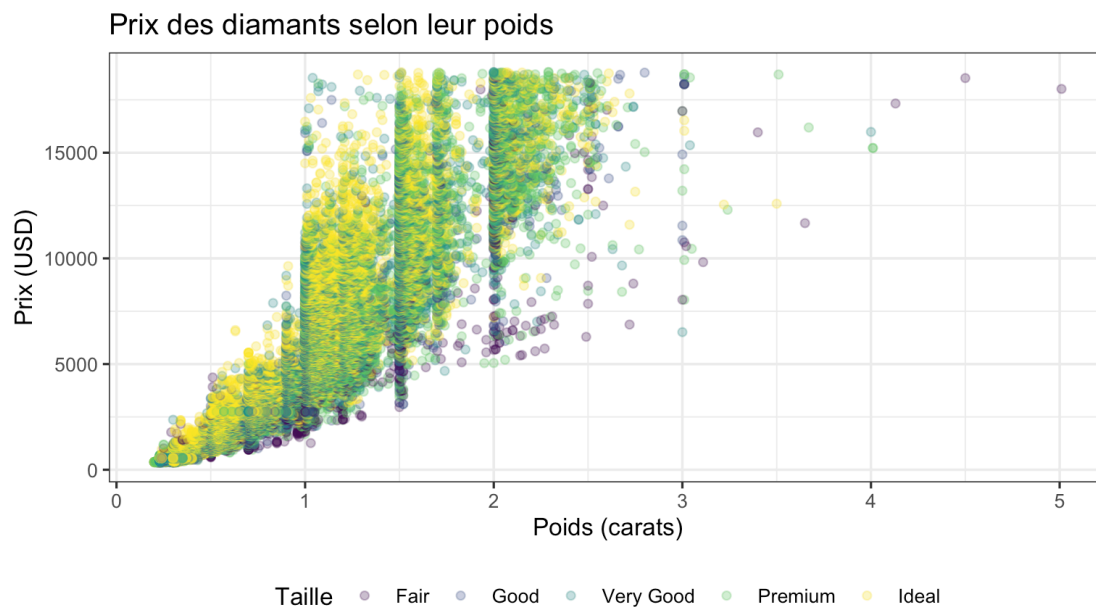


`col = iris$Species` utilise le **numéro du niveau** : setosa = 1, versicolor = 2, virginica = 3.

`levels(iris$Species)` donne les noms **dans ce même ordre** : la légende ne peut pas se tromper.

4. Les diamants avec **ggplot2**

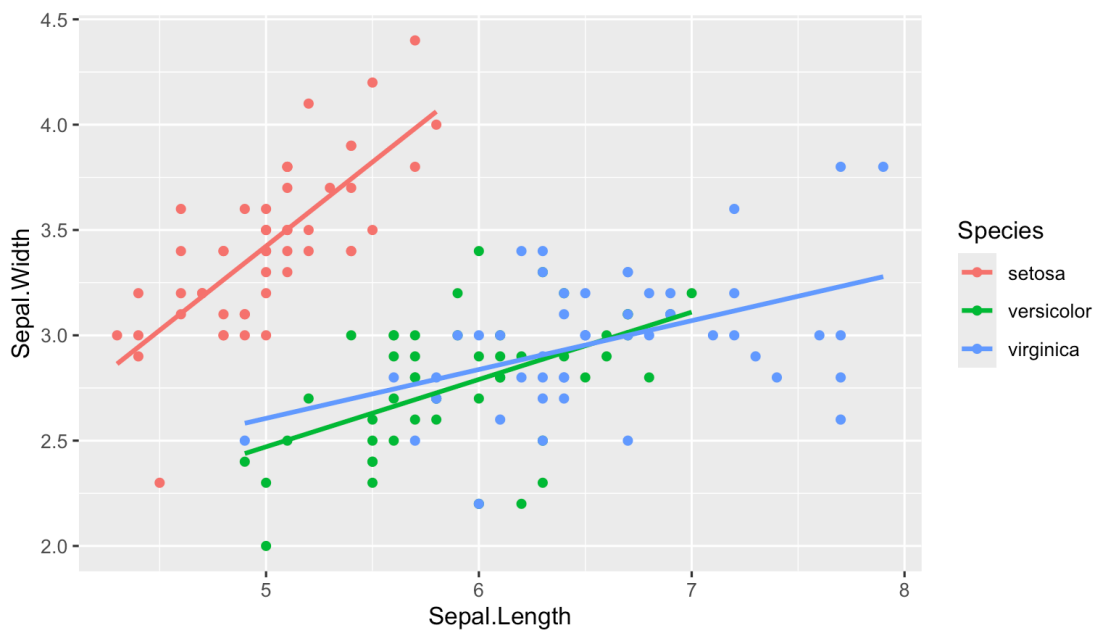
```
p4 <- ggplot(diamonds, aes(x = carat, y = price, color = cut)) +
  geom_point(alpha = 0.3) +
  labs(title = "Prix des diamants selon leur poids", x = "Poids (carats)", y = "Prix (USD)") +
  theme_bw() +
  theme(legend.position = "bottom")
p4
```



53 940 points se superposent : `alpha` (transparence) aide à voir où ils sont nombreux.

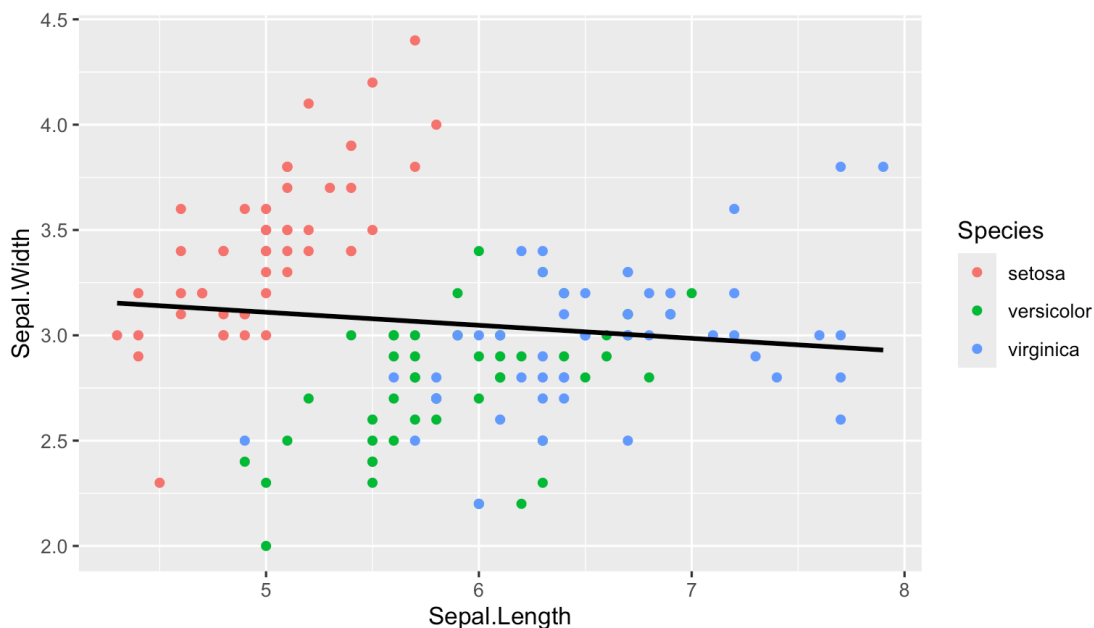
5. Une ou trois droites

```
ggplot(iris, aes(Sepal.Length, Sepal.Width, color = Species)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
#> `geom_smooth()` using formula = 'y ~ x'
```



`color = Species` est dans `ggplot()` : il vaut pour **toutes** les couches, et `geom_smooth()` ajuste **une droite par espèce**. Pour une seule droite, on ne colore que les points :

```
ggplot(iris, aes(Sepal.Length, Sepal.Width)) +
  geom_point(aes(color = Species)) +
  geom_smooth(method = "lm", se = FALSE, color = "black")
#> `geom_smooth()` using formula = 'y ~ x'
```

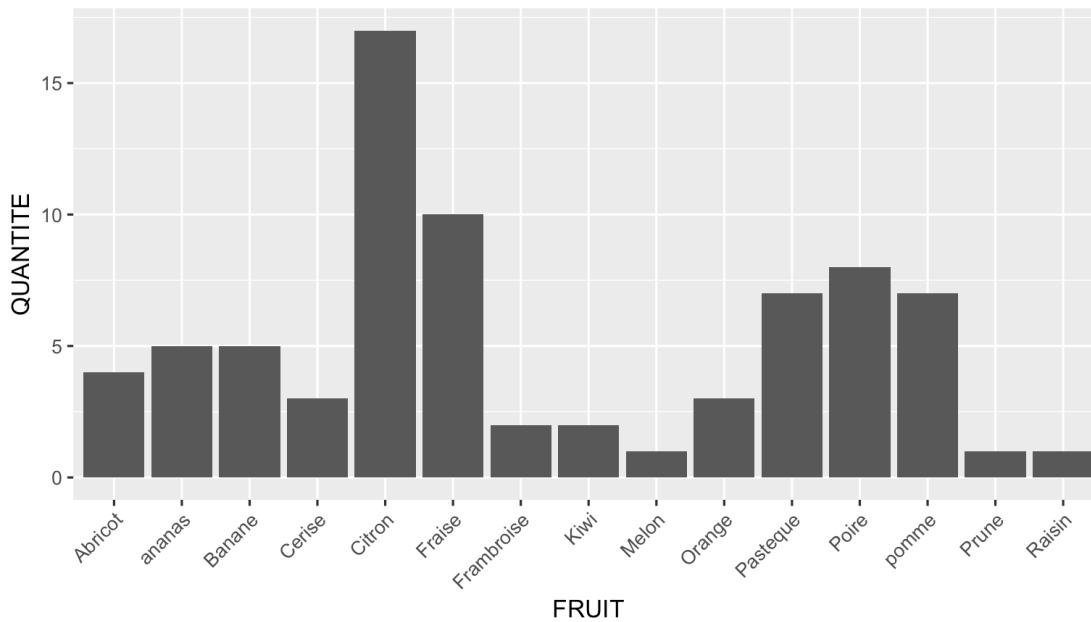


Les deux sont justes, mais racontent des histoires opposées : dans chaque espèce, les sépales longs sont plus larges (pentes positives) ; toutes espèces confondues, la pente est légèrement **négative**. C'est un paradoxe de Simpson (module 9).

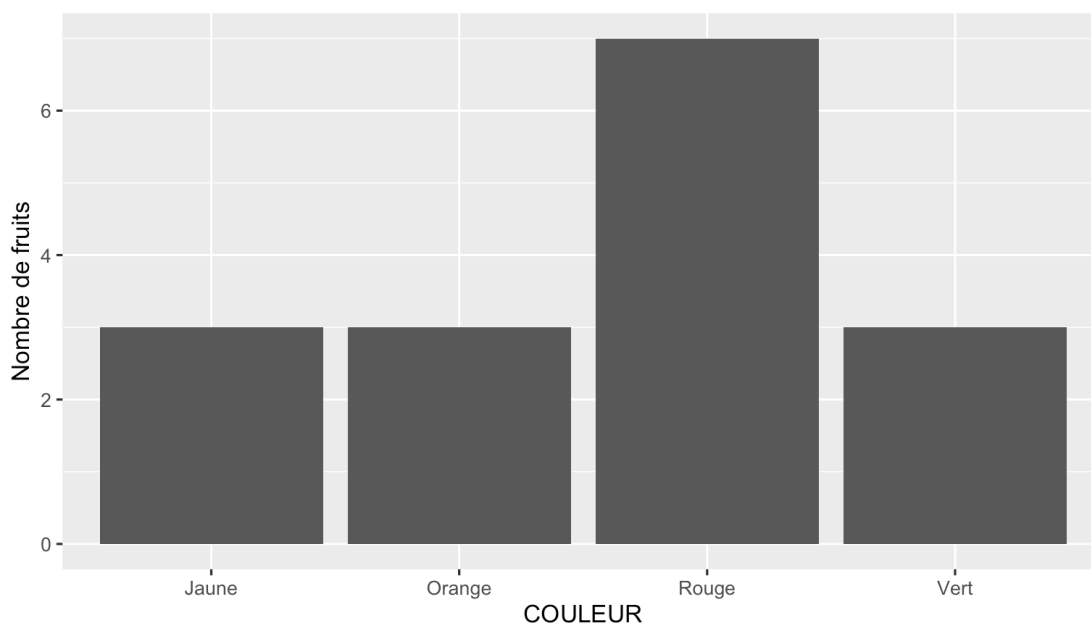
6. Les fruits

```
fruit <- read_tsv("/home/public/visu/fruit.tsv")
```

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) +  
  geom_col() +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
ggplot(fruit, aes(x = COULEUR)) +  
  geom_bar() +  
  labs(y = "Nombre de fruits")
```



Remarquez l'ordre des fruits : alphabétique, sans tenir compte des majuscules (**ananas** après **Abricot** , **pomme** après **Poire**). La saisie n'est pas homogène (**ananas** , **pomme** , et une **Frambroise**) : à nettoyer avant un vrai rapport (module 5).

7. Sauvegarder

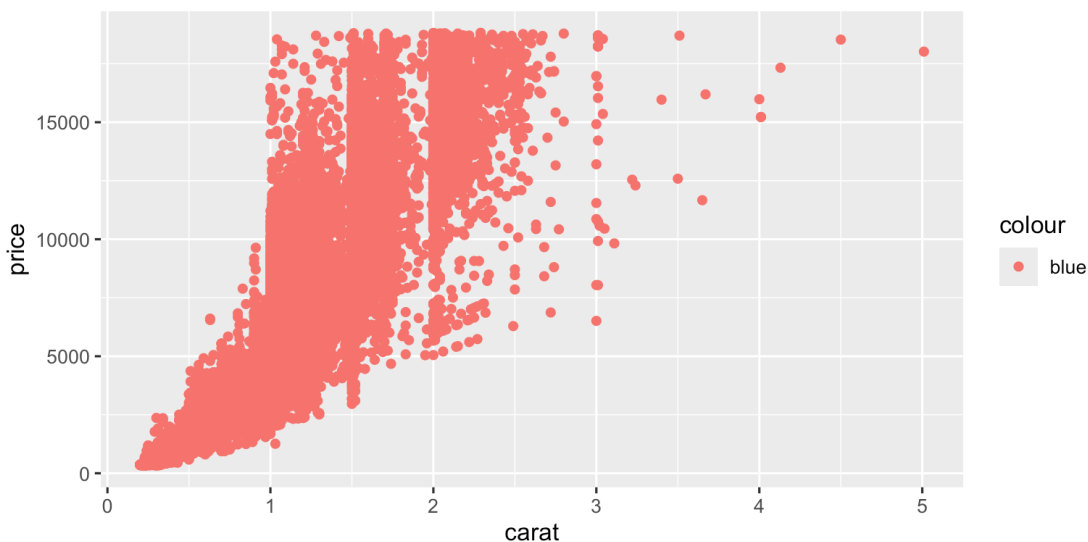
```
ggsave("diamants.png", plot = p4, width = 16, height = 10, units = "cm", dpi = 300)
```

Sans `plot =` , `ggsave()` enregistre le **dernier graphique affiché**, ici celui des fruits.

Et si... ?

1. Une constante dans `aes()`

```
ggplot(diamonds, aes(carat, price, color = cut)) +  
  geom_point(aes(color = "blue"))
```



Les points sont **rouge saumon**, et la légende annonce « blue ». Dans `aes()` , `"blue"` est une **donnée** (une colonne qui vaudrait `"blue"` partout) : `ggplot2` lui donne la première couleur de sa palette. Elle remplace aussi la couleur selon `cut` . Une couleur fixe s'écrit **hors** de `aes()` : `geom_point(color = "blue")` .

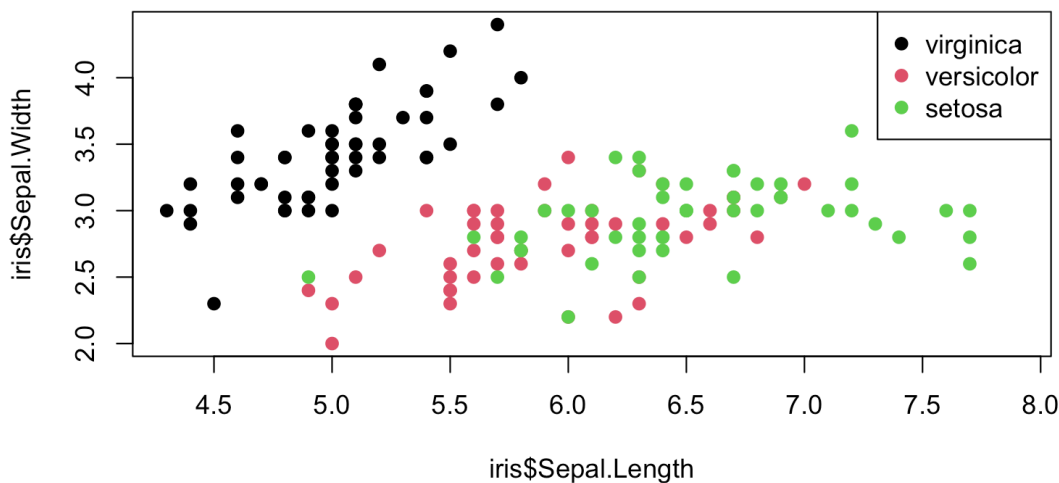
2. Une colonne hors de `aes()`

```
ggplot(diamonds, aes(carat, price)) + geom_point(color = cut)  
#> Error in `geom_point()` :  
#> ! Problem while setting up geom aesthetics.  
#> i Error occurred in the 1st layer.  
#> Caused by error in `list_sizes()` :  
#> ! `x$colour` must be a vector, not a function.
```

Hors de `aes()`, R cherche un **objet** `cut` dans l'environnement, pas une colonne du tableau. Il en trouve un : la **fonction** `cut()` de R (qui découpe un vecteur en classes), d'où ce message déroutant, ``x$colour` must be a vector, not a function`. Avec une colonne au nom moins courant, comme `Species`, le message serait `object 'Species' not found`.

3. Une légende dans le désordre

```
plot(iris$Sepal.Length, iris$Sepal.Width, col = iris$Species, pch = 19)
legend("topright", legend = c("virginica", "versicolor", "setosa"), col = 1:3, pch = 19)
```



Aucun message : la légende est un texte libre, que R ne compare pas aux données. Les points noirs (en haut à gauche) sont des *setosa*, mais la légende dit « virginica ». C'est l'avantage de `ggplot2` : la légende est construite à partir des données.

4. `geom_bar()` avec un `y`

```
ggplot(fruit, aes(x = FRUIT, y = QUANTITE)) + geom_bar()
#> Error in `geom_bar()` :
#> ! Problem while computing stat.
#> i Error occurred in the 1st layer.
#> Caused by error in `setup_params()` :
#> ! `stat_count()` must only have an x or y aesthetic.
```

`geom_bar()` **calcule** la hauteur (il compte les lignes) : il refuse qu'on la lui donne. `geom_col()`, ou l'ancienne forme `geom_bar(stat = "identity")`, tracent `y` tel quel.

5. Citron et Rouge

```

filter(fruit, FRUIT == "Citron")
#> # A tibble: 2 × 3
#>   FRUIT COULEUR QUANTITE
#>   <chr>  <chr>      <dbl>
#> 1 Citron Jaune          9
#> 2 Citron Vert           8
count(fruit, COULEUR)
#> # A tibble: 4 × 2
#>   COULEUR     n
#>   <chr>   <int>
#> 1 Jaune     3
#> 2 Orange    3
#> 3 Rouge     7
#> 4 Vert      3

```

- La barre « Citron » vaut **17** : deux lignes ont `FRUIT == "Citron"` (jaune et vert), et `geom_col()` les **empile**. Rien ne le signale : la barre a l'air d'une seule valeur.
- La barre « Rouge » vaut **7** : `geom_bar()` compte les **lignes** rouges (7 fruits), sans lire `QUANTITE` . Pour la quantité totale de fruits rouges : `geom_col()` avec `y = QUANTITE` (31), ou `summarise()` d'abord.